CESI
ÉCOLE D'INGÉNIEURS

Projet CHU – Entrepôt de données



Livrable 2 | Rapport de qualité des données

Vendredi 6 novembre 2026

A. Tellez
M. Mazou
M. Sepulveda
N. Durtelle de Saint Sauveur

Fisa Info 25–28
CESI

TABLE DES MATIÈRES

1 Objectif du contrôle **2**

2 Contrôle de complétude des données **2**
2.1 Méthodologie 2
2.2 Requêtes de contrôle 3
2.2.1 DIM_Etablissement 4
2.2.2 DIM_Professionnel 6
2.2.3 DIM_Temps 8
2.2.4 DIM_Patient 9
2.2.5 DIM_Diagnostic 10
2.2.6 DIM_Region_INSEE 12
2.2.7 FAIT_SATISFACTION 13
2.2.8 FAIT_DECES 15
2.2.9 FAIT_CONSULTATION 16
2.2.10 FAIT_HOSPITALISATION 18
2.3 Résultats attendus 19

3 Contrôle de cohérence référentielle **20**
3.1 Méthodologie 20
3.2 Vérification des établissements FINESS 20
3.3 Vérification des clés étrangères 22
3.3.1 Attention : référentiel des régions de FAIT_DECES 28
3.4 Résultats attendus 29

4 Détection et traitement des doublons **30**
4.1 Méthodologie 31
4.2 Requêtes de détection 31
4.3 Traitement des doublons 35

5 Synthèse de la qualité des données **36**

6 Conclusion **36**

1 OBJECTIF DU CONTRÔLE

L'objectif de ce rapport est d'évaluer la qualité des données intégrées dans l'entrepôt de données Hive, à la suite de leur extraction depuis PostgreSQL et de leur transformation via Talend.

Trois axes de contrôle sont étudiés :

- **Complétude des données** : mesure des valeurs renseignées et des valeurs manquantes pour chaque attribut.
- **Cohérence référentielle** : vérification de la correspondance entre les clés étrangères des tables de faits et les identifiants des dimensions, notamment les codes FINESS.
- **Détection et traitement des doublons** : identification des enregistrements dupliqués et définition des méthodes de correction.

Les contrôles sont effectués sur les tables internes de l'entrepôt, stockées au format Parquet.

2 CONTRÔLE DE COMPLÉTUDE DES DONNÉES

2.1 MÉTHODOLOGIE

Le contrôle de complétude consiste à identifier les attributs dont les valeurs sont manquantes (NULL).

Pour chaque attribut, trois indicateurs sont calculés :

- Le nombre total d'enregistrements.
- Le nombre de valeurs renseignées et le nombre de valeurs manquantes.
- Le taux de complétude exprimé en pourcentage.

Formule de calcul :

$$\text{Taux de complétude} = (\text{Nombre de valeurs renseignées} / \text{Nombre total d'enregistrements}) \times 100$$

Un taux de 100 % indique que toutes les lignes possèdent une valeur pour l'attribut étudié.

2.2 REQUÊTES DE CONTRÔLE

Les contrôles sont réalisés sur les principales dimensions et tables de faits.

Exemple : DIM_Patient

```
SET mapreduce.map.memory.mb=1024;
SET mapreduce.reduce.memory.mb=1024;

SET mapreduce.map.java.opts=-Xmx768m;
SET mapreduce.reduce.java.opts=-Xmx768m;

SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Patient) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Patient) AS Valeurs_Null,
    ROUND(COUNT(Id_Patient) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Patient

UNION ALL

SELECT
    'Sexe' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Sexe) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Sexe) AS Valeurs_Null,
    ROUND(COUNT(Sexe) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Patient

UNION ALL

SELECT
    'Age' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Age) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Age) AS Valeurs_Null,
    ROUND(COUNT(Age) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Patient
```

Cette requête permet de mesurer la complétude des attributs de DIM_Patient. On peut donc vérifier s'il y a des valeurs NULL et leur taux de complétude

Le même contrôle doit être reproduit pour les attributs suivants :

Table	Attributs à contrôler	Pourquoi ?
DIM_Etablissement	Id_Etablissement, finess_site, raison_sociale_site, code_postal, commune	Identification et localisation
DIM_Professionnel	Id_Professionnel, identifiant, nom, prenom, profession	Identification du professionnel
DIM_Temps	Id_Temps, Date, Annee	Cohérence temporelle
DIM_Patient	Id_Patient, Sexe, Age	Informations démographiques
DIM_Diagnostic	Id_Diagnostic, Code_Diagnostic, Diagnostic	Identification du diagnostic
DIM_Region_INSEE	Id_Region, Code_Commune_INSEE, Code_Region, Nom_Region	Identification géographique
FAIT_SATISFACTION	Id_Satisfaction, Id_Etablissement, Id_Region, Score	Identification et indicateurs
FAIT_DECES	Id_Deces, Id_Temps, Id_Region_Deces, Age_Deces	Identification et données de décès
FAIT_CONSULTATION	num_consultation, Id_Patient, Id_Temps, Id_Professionnel, Id_Etablissement	Identification et relations
FAIT_HOSPITALISATION	Id_Hospitalisation, Id_Patient, Id_Etablissement, Id_Date_Entree	Identification et relations

Remarque : les attributs qui doivent obligatoirement être renseignés doivent être distingués de ceux qui peuvent légitimement être NULL selon les règles métier.

Les requêtes ci-dessous ont été exécutées avec beeline sur la base `htables` de la VM Cloudera (Hive 1.1), le 6 novembre 2026.

Les scripts complets sont dans le dépôt : `qualite/01_completude.hql`, `qualite/02_coherence_referentielle.hql` et `qualite/03_doublons.hql`.

Chaque script commence par `USE htables;` et par les quatre paramètres mémoire de l'exemple ci-dessus.

Les nombres sont écrits avec une espace pour les milliers (33 911). Les taux sont arrondis à deux décimales par la requête.

Le contrôle de DIM_Patient est reproduit pour toutes les tables du tableau, avec exactement les attributs listés.

Remarque : `COUNT(colonne)` ne compte pas les valeurs NULL, mais il compte les chaînes vides `''`. Pour les colonnes texte, une seconde requête compte donc les valeurs vides. Elle n'est commentée que lorsque le nombre n'est pas nul.

2.2.1 DIM_Etablissement

```
SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Etablissement) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Etablissement) AS Valeurs_Null,
    ROUND(COUNT(Id_Etablissement) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Etablissement

UNION ALL
```

```sql
SELECT
    'finess_site' AS Colonne,
    COUNT(*) AS Total,
    COUNT(finess_site) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(finess_site) AS Valeurs_Null,
    ROUND(COUNT(finess_site) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Etablissement

UNION ALL

SELECT
    'raison_sociale_site' AS Colonne,
    COUNT(*) AS Total,
    COUNT(raison_sociale_site) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(raison_sociale_site) AS Valeurs_Null,
    ROUND(COUNT(raison_sociale_site) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Etablissement

UNION ALL

SELECT
    'code_postal' AS Colonne,
    COUNT(*) AS Total,
    COUNT(code_postal) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(code_postal) AS Valeurs_Null,
    ROUND(COUNT(code_postal) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Etablissement

UNION ALL

SELECT
    'commune' AS Colonne,
    COUNT(*) AS Total,
    COUNT(commune) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(commune) AS Valeurs_Null,
    ROUND(COUNT(commune) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Etablissement;

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Etablissement
WHERE TRIM(CAST(Id_Etablissement AS STRING)) = ''

UNION ALL

SELECT
    'finess_site' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Etablissement
WHERE TRIM(finess_site) = ''

UNION ALL
```

```sql
SELECT
    'raison_sociale_site' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Etablissement
WHERE TRIM(raison_sociale_site) = ''

UNION ALL

SELECT
    'code_postal' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Etablissement
WHERE TRIM(code_postal) = ''

UNION ALL

SELECT
    'commune' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Etablissement
WHERE TRIM(commune) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Etablissement	33911	33911	0	0	100.0
finess_site	33911	33911	0	33711	0.59
raison_sociale_site	33911	33911	0	0	100.0
code_postal	33911	33911	0	2	99.99
commune	33911	33911	0	140	99.59

Aucune valeur NULL. Mais finess_site est vide (`''`) sur 33 711 lignes sur 33 911 : seuls 200 établissements ont un FINESS géographique, soit un taux réel de 0,59 %. Le fichier chargé sur HDFS (`/user/cloudera/etablissement/`) a déjà ce champ vide : le défaut est en amont de Hive (source PostgreSQL ou export Talend), pas dans le chargement HiveQL. Il y a aussi 2 code_postal vides et 140 communes vides.

2.2.2 DIM_Professionnel

```sql
SELECT
    'Id_Professionnel' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Professionnel) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Professionnel) AS Valeurs_Null,
    ROUND(COUNT(Id_Professionnel) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Professionnel

UNION ALL

SELECT
    'identifiant' AS Colonne,
    COUNT(*) AS Total,
    COUNT(identifiant) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(identifiant) AS Valeurs_Null,
    ROUND(COUNT(identifiant) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Professionnel
```

```sql
UNION ALL

SELECT
    'nom' AS Colonne,
    COUNT(*) AS Total,
    COUNT(nom) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(nom) AS Valeurs_Null,
    ROUND(COUNT(nom) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Professionnel

UNION ALL

SELECT
    'prenom' AS Colonne,
    COUNT(*) AS Total,
    COUNT(prenom) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(prenom) AS Valeurs_Null,
    ROUND(COUNT(prenom) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Professionnel

UNION ALL

SELECT
    'profession' AS Colonne,
    COUNT(*) AS Total,
    COUNT(profession) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(profession) AS Valeurs_Null,
    ROUND(COUNT(profession) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Professionnel;

SELECT
    'Id_Professionnel' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Professionnel
WHERE TRIM(CAST(Id_Professionnel AS STRING)) = ''

UNION ALL

SELECT
    'identifiant' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Professionnel
WHERE TRIM(identifiant) = ''

UNION ALL

SELECT
    'nom' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Professionnel
WHERE TRIM(nom) = ''

UNION ALL
```

```sql
SELECT
    'prenom' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Professionnel
WHERE TRIM(prenom) = ''

UNION ALL

SELECT
    'profession' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Professionnel
WHERE TRIM(profession) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Professionnel	1048572	1048572	0	0	100.0
identifiant	1048572	1048572	0	0	100.0
nom	1048572	1048572	0	0	100.0
prenom	1048572	1048572	0	17	100.0 (99,998 avant arrondi)
profession	1048572	1048572	0	0	100.0

Aucune valeur NULL. On compte 17 prénoms vides sur 1 048 572 professionnels, ce qui est négligeable.

2.2.3 DIM_Temps

```sql
SELECT
    'Id_Temps' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Temps) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Temps) AS Valeurs_Null,
    ROUND(COUNT(Id_Temps) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Temps

UNION ALL

SELECT
    'Date' AS Colonne,
    COUNT(*) AS Total,
    COUNT(`Date`) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(`Date`) AS Valeurs_Null,
    ROUND(COUNT(`Date`) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Temps

UNION ALL

SELECT
    'Annee' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Annee) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Annee) AS Valeurs_Null,
    ROUND(COUNT(Annee) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Temps;
```

```sql
SELECT
    'Id_Temps' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Temps
WHERE TRIM(CAST(Id_Temps AS STRING)) = ''

UNION ALL

SELECT
    'Date' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Temps
WHERE TRIM(`Date`) = ''

UNION ALL

SELECT
    'Annee' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Temps
WHERE TRIM(CAST(Annee AS STRING)) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Temps	55151	55151	0	0	100.0
Date	55151	55151	0	0	100.0
Annee	55151	55151	0	0	100.0

Aucune valeur NULL ni vide : la dimension temps est complète.

2.2.4 DIM_Patient

```sql
SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Patient) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Patient) AS Valeurs_Null,
    ROUND(COUNT(Id_Patient) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Patient

UNION ALL

SELECT
    'Sexe' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Sexe) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Sexe) AS Valeurs_Null,
    ROUND(COUNT(Sexe) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Patient

UNION ALL
```

```sql
SELECT
    'Age' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Age) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Age) AS Valeurs_Null,
    ROUND(COUNT(Age) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Patient;

SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Patient
WHERE TRIM(CAST(Id_Patient AS STRING)) = ''

UNION ALL

SELECT
    'Sexe' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Patient
WHERE TRIM(Sexe) = ''

UNION ALL

SELECT
    'Age' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Patient
WHERE TRIM(CAST(Age AS STRING)) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_Null	Valeurs_ Vides	Taux_Completude
Id_Patient	99999	99999	0	0	100.0
Sexe	99999	99999	0	0	100.0
Age	99999	99999	0	0	100.0

Aucune valeur NULL ni vide : le taux de complétude, valeurs vides comprises, est de 100 % et les informations démographiques sont complètes.

2.2.5 DIM_Diagnostic

```sql
SELECT
    'Id_Diagnostic' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Diagnostic) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Diagnostic) AS Valeurs_Null,
    ROUND(COUNT(Id_Diagnostic) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Diagnostic

UNION ALL
```

```sql
SELECT
    'Code_Diagnostic' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Code_Diagnostic) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Code_Diagnostic) AS Valeurs_Null,
    ROUND(COUNT(Code_Diagnostic) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Diagnostic

UNION ALL

SELECT
    'Diagnostic' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Diagnostic) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Diagnostic) AS Valeurs_Null,
    ROUND(COUNT(Diagnostic) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Diagnostic;

SELECT
    'Id_Diagnostic' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Diagnostic
WHERE TRIM(CAST(Id_Diagnostic AS STRING)) = ''

UNION ALL

SELECT
    'Code_Diagnostic' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Diagnostic
WHERE TRIM(Code_Diagnostic) = ''

UNION ALL

SELECT
    'Diagnostic' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Diagnostic
WHERE TRIM(Diagnostic) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Diagnostic	15489	15489	0	0	100.0
Code_Diagnostic	15489	15489	0	0	100.0
Diagnostic	15489	15489	0	0	100.0

Aucune valeur NULL ni vide : chaque diagnostic a un code et un libellé.

2.2.6 DIM_Region_INSEE

```
SELECT
    'Id_Region' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Region) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Region) AS Valeurs_Null,
    ROUND(COUNT(Id_Region) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Region_INSEE

UNION ALL

SELECT
    'Code_Commune_INSEE' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Code_Commune_INSEE) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Code_Commune_INSEE) AS Valeurs_Null,
    ROUND(COUNT(Code_Commune_INSEE) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Region_INSEE

UNION ALL

SELECT
    'Code_Region' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Code_Region) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Code_Region) AS Valeurs_Null,
    ROUND(COUNT(Code_Region) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Region_INSEE

UNION ALL

SELECT
    'Nom_Region' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Nom_Region) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Nom_Region) AS Valeurs_Null,
    ROUND(COUNT(Nom_Region) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM DIM_Region_INSEE;

SELECT
    'Id_Region' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Region_INSEE
WHERE TRIM(CAST(Id_Region AS STRING)) = ''

UNION ALL

SELECT
    'Code_Commune_INSEE' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Region_INSEE
WHERE TRIM(Code_Commune_INSEE) = ''

UNION ALL
```

```sql
SELECT
    'Code_Region' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Region_INSEE
WHERE TRIM(Code_Region) = ''

UNION ALL

SELECT
    'Nom_Region' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM DIM_Region_INSEE
WHERE TRIM(Nom_Region) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Region	39520	39520	0	0	100.0
Code_Commune_INSEE	39520	39520	0	0	100.0
Code_Region	39520	39520	0	358	99.09
Nom_Region	39520	39520	0	358	99.09

Aucune valeur NULL. Code_Region et Nom_Region sont vides sur 358 lignes (0,91 %). Ces 358 lignes sont exactement les 254 lignes de type Etranger et les 104 lignes de type COM (collectivités d'outre-mer) : elles n'ont pas de région française, le vide est donc normal.

2.2.7 FAIT_SATISFACTION

```sql
SELECT
    'Id_Satisfaction' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Satisfaction) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Satisfaction) AS Valeurs_Null,
    ROUND(COUNT(Id_Satisfaction) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_SATISFACTION

UNION ALL

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Etablissement) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Etablissement) AS Valeurs_Null,
    ROUND(COUNT(Id_Etablissement) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_SATISFACTION

UNION ALL

SELECT
    'Id_Region' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Region) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Region) AS Valeurs_Null,
    ROUND(COUNT(Id_Region) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_SATISFACTION
```

```sql
UNION ALL

SELECT
    'Score' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Score) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Score) AS Valeurs_Null,
    ROUND(COUNT(Score) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_SATISFACTION;

SELECT
    'Id_Satisfaction' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_SATISFACTION
WHERE TRIM(CAST(Id_Satisfaction AS STRING)) = ''

UNION ALL

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_SATISFACTION
WHERE TRIM(CAST(Id_Etablissement AS STRING)) = ''

UNION ALL

SELECT
    'Id_Region' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_SATISFACTION
WHERE TRIM(CAST(Id_Region AS STRING)) = ''

UNION ALL

SELECT
    'Score' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_SATISFACTION
WHERE TRIM(CAST(Score AS STRING)) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Satisfaction	2080	2080	0	0	100.0
Id_Etablissement	2080	2080	0	0	100.0
Id_Region	2080	2080	0	0	100.0
Score	2080	1624	456	0	78.08

Score n'est renseigné que sur 1 624 lignes : 456 lignes sont NULL (taux de 78,08 %). C'est l'attribut le moins complet de l'entrepôt. Un score NULL correspond à un établissement sans résultat publié : c'est une valeur qui peut légitimement être NULL, il ne faut pas la remplacer par 0.

2.2.8 FAIT_DECES

```
SELECT
    'Id_Deces' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Deces) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Deces) AS Valeurs_Null,
    ROUND(COUNT(Id_Deces) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_DECES

UNION ALL

SELECT
    'Id_Temps' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Temps) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Temps) AS Valeurs_Null,
    ROUND(COUNT(Id_Temps) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_DECES

UNION ALL

SELECT
    'Id_Region_Deces' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Region_Deces) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Region_Deces) AS Valeurs_Null,
    ROUND(COUNT(Id_Region_Deces) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_DECES

UNION ALL

SELECT
    'Age_Deces' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Age_Deces) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Age_Deces) AS Valeurs_Null,
    ROUND(COUNT(Age_Deces) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_DECES;

SELECT
    'Id_Deces' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_DECES
WHERE TRIM(CAST(Id_Deces AS STRING)) = ''

UNION ALL

SELECT
    'Id_Temps' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_DECES
WHERE TRIM(CAST(Id_Temps AS STRING)) = ''

UNION ALL
```

```sql
SELECT
    'Id_Region_Deces' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_DECES
WHERE TRIM(CAST(Id_Region_Deces AS STRING)) = ''

UNION ALL

SELECT
    'Age_Deces' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_DECES
WHERE TRIM(CAST(Age_Deces AS STRING)) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Deces	24875113	24875113	0	0	100.0
Id_Temps	24875113	24875113	0	0	100.0
Id_Region_Deces	24875113	24875113	0	0	100.0
Age_Deces	24875113	24874527	586	0	100.0 (99,998 avant arrondi)

Les clés sont toutes renseignées. Age_Deces est NULL sur 586 lignes sur 24 875 113 (99,998 %, affiché 100.0 après arrondi).

2.2.9 FAIT_CONSULTATION

```sql
SELECT
    'num_consultation' AS Colonne,
    COUNT(*) AS Total,
    COUNT(num_consultation) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(num_consultation) AS Valeurs_Null,
    ROUND(COUNT(num_consultation) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_CONSULTATION

UNION ALL

SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Patient) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Patient) AS Valeurs_Null,
    ROUND(COUNT(Id_Patient) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_CONSULTATION

UNION ALL

SELECT
    'Id_Temps' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Temps) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Temps) AS Valeurs_Null,
    ROUND(COUNT(Id_Temps) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_CONSULTATION

UNION ALL
```

```sql
SELECT
    'Id_Professionnel' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Professionnel) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Professionnel) AS Valeurs_Null,
    ROUND(COUNT(Id_Professionnel) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_CONSULTATION

UNION ALL

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Etablissement) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Etablissement) AS Valeurs_Null,
    ROUND(COUNT(Id_Etablissement) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_CONSULTATION;

SELECT
    'num_consultation' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_CONSULTATION
WHERE TRIM(CAST(num_consultation AS STRING)) = ''

UNION ALL

SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_CONSULTATION
WHERE TRIM(CAST(Id_Patient AS STRING)) = ''

UNION ALL

SELECT
    'Id_Temps' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_CONSULTATION
WHERE TRIM(CAST(Id_Temps AS STRING)) = ''

UNION ALL

SELECT
    'Id_Professionnel' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_CONSULTATION
WHERE TRIM(CAST(Id_Professionnel AS STRING)) = ''

UNION ALL

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_CONSULTATION
WHERE TRIM(CAST(Id_Etablissement AS STRING)) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
num_consultation	1027136	1027136	0	0	100.0
Id_Patient	1027136	1027136	0	0	100.0
Id_Temps	1027136	1027136	0	0	100.0
Id_Professionnel	1027136	1027136	0	0	100.0
Id_Etablissement	1027136	1027136	0	0	100.0

Toutes les clés sont renseignées. Attention : renseigné ne veut pas dire valide. Id_Etablissement vaut -1 sur une grande partie des lignes (voir 3.3).

2.2.10 FAIT_HOSPITALISATION

```
SELECT
    'Id_Hospitalisation' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Hospitalisation) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Hospitalisation) AS Valeurs_Null,
    ROUND(COUNT(Id_Hospitalisation) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_HOSPITALISATION

UNION ALL

SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Patient) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Patient) AS Valeurs_Null,
    ROUND(COUNT(Id_Patient) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_HOSPITALISATION

UNION ALL

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Etablissement) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Etablissement) AS Valeurs_Null,
    ROUND(COUNT(Id_Etablissement) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_HOSPITALISATION

UNION ALL

SELECT
    'Id_Date_Entree' AS Colonne,
    COUNT(*) AS Total,
    COUNT(Id_Date_Entree) AS Valeurs_Renseignees,
    COUNT(*) - COUNT(Id_Date_Entree) AS Valeurs_Null,
    ROUND(COUNT(Id_Date_Entree) * 100.0 / COUNT(*), 2) AS Taux_Completude
FROM FAIT_HOSPITALISATION;

SELECT
    'Id_Hospitalisation' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_HOSPITALISATION
WHERE TRIM(CAST(Id_Hospitalisation AS STRING)) = ''
```

```sql
UNION ALL

SELECT
    'Id_Patient' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_HOSPITALISATION
WHERE TRIM(CAST(Id_Patient AS STRING)) = ''

UNION ALL

SELECT
    'Id_Etablissement' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_HOSPITALISATION
WHERE TRIM(CAST(Id_Etablissement AS STRING)) = ''

UNION ALL

SELECT
    'Id_Date_Entree' AS Colonne,
    COUNT(*) AS Valeurs_Vides
FROM FAIT_HOSPITALISATION
WHERE TRIM(CAST(Id_Date_Entree AS STRING)) = '';
```

Colonne	Total	Valeurs_ Renseignees	Valeurs_ Null	Valeurs_ Vides	Taux_ Completude
Id_Hospitalisation	2478	2478	0	0	100.0
Id_Patient	2478	2478	0	0	100.0
Id_Etablissement	2478	2478	0	0	100.0
Id_Date_Entree	2478	2478	0	0	100.0

Toutes les clés sont renseignées. Comme pour les consultations, la validité des clés est vérifiée en 3.3.

2.3 RÉSULTATS ATTENDUS

Les résultats seront regroupés dans un tableau de synthèse :

Table	Attribut	Taux de complétude
DIM_Etablissement	finess_site	100 % de valeurs non NULL, mais 33 711 valeurs vides : taux réel 0,59 % (200 / 33 911)
DIM_Etablissement	raison_sociale_site	100 %
DIM_Professionnel	identifiant	100 %
DIM_Patient	Sexe	100 %
DIM_Diagnostic	Code_Diagnostic	100 %
DIM_Temps	Date	100 %

Les résultats permettront d'identifier les attributs les moins renseignés et de déterminer si les données manquantes nécessitent un traitement.

Les attributs les moins renseignés sont finess_site (0,59 % réellement renseigné), Score de FAIT_SATISFACTION (78,08 %) et Code_Region / Nom_Region de DIM_Region_INSEE (99,09 % une fois les vides retirés, vides normaux pour l'étranger et les COM). Seul finess_site demande un traitement : il ne peut pas servir de clé d'identification, il faut utiliser finess_etablissement_juridique ou Id_Etablissement.

3 CONTRÔLE DE COHÉRENCE RÉFÉRENTIELLE

3.1 MÉTHODOLOGIE

La cohérence référentielle consiste à vérifier que les identifiants utilisés dans les tables de faits correspondent bien à des enregistrements existants dans les dimensions.

Une référence est considérée comme invalide lorsqu'une clé étrangère présente dans une table de faits ne possède aucune correspondance dans sa dimension associée.

Une attention particulière est portée aux identifiants FINESS des établissements de santé.

3.2 VÉRIFICATION DES ÉTABLISSEMENTS FINESS

Les établissements sont identifiés dans DIM_Etablissement grâce aux attributs finess_site et finess_etablissement_juridique.

Le contrôle permet de détecter les identifiants FINESS manquants ou invalides.

Exemple de vérification du format FINESS :

```
SELECT
    COUNT(*) AS Total_Invalide
FROM DIM_Etablissement
WHERE finess_site IS NOT NULL
AND NOT (finess_site RLIKE '^[0-9]{9}$');
```

Cette requête vérifie que les identifiants FINESS géographiques renseignés comportent neuf chiffres.

Format : un FINESS valide comporte neuf chiffres. La requête du rapport est appliquée aux deux colonnes FINESS.

```
SELECT
COUNT(*) AS Total_Invalide
FROM DIM_Etablissement
WHERE finess_site IS NOT NULL
AND NOT (finess_site RLIKE '^[0-9]{9}$');
```

```sql
SELECT
COUNT(*) AS Total_Invalide
FROM DIM_Etablissement
WHERE finess_etablissement_juridique IS NOT NULL
AND NOT (finess_etablissement_juridique RLIKE '^[0-9]{9}$');
```

Colonne	Total_ Invalide
finess_site	33711
finess_etablissement_juridique	209

Détail des valeurs invalides, valeurs manquantes et unicité de finess_site :

```sql
SELECT
'finess_site' AS Colonne,
COUNT(*) AS Total_Invalide,
SUM(CASE WHEN finess_site RLIKE '^2[AB][0-9]{7}$' THEN 1 ELSE 0 END) AS Dont_Corse,
SUM(CASE WHEN TRIM(finess_site) = '' THEN 1 ELSE 0 END) AS Dont_Vide
FROM DIM_Etablissement
WHERE finess_site IS NOT NULL
AND NOT (finess_site RLIKE '^[0-9]{9}$')

UNION ALL

SELECT
'finess_etablissement_juridique' AS Colonne,
COUNT(*) AS Total_Invalide,
SUM(CASE WHEN finess_etablissement_juridique RLIKE '^2[AB][0-9]{7}$' THEN 1 ELSE 0
↪  END) AS Dont_Corse,
SUM(CASE WHEN TRIM(finess_etablissement_juridique) = '' THEN 1 ELSE 0 END) AS
↪  Dont_Vide
FROM DIM_Etablissement
WHERE finess_etablissement_juridique IS NOT NULL
AND NOT (finess_etablissement_juridique RLIKE '^[0-9]{9}$');

SELECT
'finess_site' AS Colonne,
COUNT(*) AS Total,
SUM(CASE WHEN finess_site IS NULL THEN 1 ELSE 0 END) AS Valeurs_Null,
SUM(CASE WHEN TRIM(finess_site) = '' THEN 1 ELSE 0 END) AS Valeurs_Vides
FROM DIM_Etablissement

UNION ALL

SELECT
'finess_etablissement_juridique' AS Colonne,
COUNT(*) AS Total,
SUM(CASE WHEN finess_etablissement_juridique IS NULL THEN 1 ELSE 0 END) AS
↪  Valeurs_Null,
SUM(CASE WHEN TRIM(finess_etablissement_juridique) = '' THEN 1 ELSE 0 END) AS
↪  Valeurs_Vides
FROM DIM_Etablissement;
```

```sql
SELECT
COUNT(*) AS Finess_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT finess_site, COUNT(*) AS Nombre
    FROM DIM_Etablissement
    WHERE finess_site IS NOT NULL
    GROUP BY finess_site
    HAVING COUNT(*) > 1
) t;
```

Colonne	Total_ Invalide	Dont_ Corse	Dont_ Vide
finess_site	33711	0	33711
finess_etablissement_juridique	209	209	0

Colonne	Total	Valeurs_Null	Valeurs_ Vides
finess_site	33911	0	33711
finess_etablissement_juridique	33911	0	0

Finess_Dupliques	Lignes_ Concernees
1	33711

Les 33 711 finess_site « invalides » sont tous des valeurs vides : aucun FINESS géographique renseigné n'a un mauvais format.

Les 209 finess_etablissement_juridique invalides sont tous des codes de Corse (2A ou 2B suivis de sept chiffres). Ce sont de vrais FINESS : la règle `^[0-9]{9}$` est trop stricte, la bonne règle est `^(2[AB]|[0-9]{2})[0-9]{7}$`. Il n'y a donc aucun FINESS juridique réellement faux.

Aucun FINESS n'est NULL. Pour l'unicité, la seule valeur répétée est la valeur vide (33 711 lignes) : les 200 finess_site renseignés sont tous uniques.

3.3 VÉRIFICATION DES CLÉS ÉTRANGÈRES

Exemple : contrôle de la référence établissement dans FAIT_CONSULTATION

```sql
SELECT
    COUNT(*) AS References_Invalides
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Etablissement e
ON f.Id_Etablissement = e.Id_Etablissement
WHERE e.Id_Etablissement IS NULL;
```

Si le résultat est égal à zéro, toutes les références contrôlées correspondent à un établissement existant.

Les mêmes contrôles seront réalisés pour les autres relations :

Table de faits	Clé étrangère	Dimension de référence
FAIT_SATISFACTION	Id_Etablissement	DIM_Etablissement
FAIT_SATISFACTION	Id_Region	DIM_Region_INSEE
FAIT_CONSULTATION	Id_Patient	DIM_Patient
FAIT_CONSULTATION	Id_Temps	DIM_Temps
FAIT_CONSULTATION	Id_Professionnel	DIM_Professionnel
FAIT_CONSULTATION	Id_Diagnostic	DIM_Diagnostic
FAIT_CONSULTATION	Id_Etablissement	DIM_Etablissement
FAIT_ HOSPITALISATION	Id_Patient	DIM_Patient
FAIT_ HOSPITALISATION	Id_Etablissement	DIM_Etablissement
FAIT_ HOSPITALISATION	Id_Diagnostic	DIM_Diagnostic
FAIT_ HOSPITALISATION	Id_Date_Entree	DIM_Temps
FAIT_DECES	Id_Temps	DIM_Temps

Attention : les références régionales de FAIT_DECES doivent être contrôlées après vérification de leur correspondance avec la structure de DIM_Region_INSEE. Il faut notamment confirmer que Id_Region_Deces et Id_Region_Naissance utilisent le même référentiel que Id_Region.

Une requête par ligne du tableau, dans le style de l'exemple du rapport. Deux colonnes ont été ajoutées pour expliquer les références invalides : `Dont_Null` (clé NULL) et `Dont_Moins_Un` (clé égale à -1, valeur « inconnu » mise par l'ETL). Quand il n'y a aucune référence invalide, ces deux sommes valent NULL dans beeline (somme sur zéro ligne), ce qui est noté 0 ici.

```
SELECT
'FAIT_SATISFACTION' AS Table_De_Faits,
'Id_Etablissement' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Etablissement IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Etablissement = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_SATISFACTION f
LEFT JOIN DIM_Etablissement e
ON f.Id_Etablissement = e.Id_Etablissement
WHERE e.Id_Etablissement IS NULL;

SELECT
'FAIT_SATISFACTION' AS Table_De_Faits,
'Id_Region' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Region IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Region = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_SATISFACTION f
LEFT JOIN DIM_Region_INSEE r
ON f.Id_Region = r.Id_Region
WHERE r.Id_Region IS NULL;
```

```sql
SELECT
'FAIT_CONSULTATION' AS Table_De_Faits,
'Id_Patient' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Patient IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Patient = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Patient p
ON f.Id_Patient = p.Id_Patient
WHERE p.Id_Patient IS NULL;

SELECT
'FAIT_CONSULTATION' AS Table_De_Faits,
'Id_Temps' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Temps IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Temps = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Temps d
ON f.Id_Temps = d.Id_Temps
WHERE d.Id_Temps IS NULL;

SELECT
'FAIT_CONSULTATION' AS Table_De_Faits,
'Id_Professionnel' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Professionnel IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Professionnel = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Professionnel pr
ON f.Id_Professionnel = pr.Id_Professionnel
WHERE pr.Id_Professionnel IS NULL;

SELECT
'FAIT_CONSULTATION' AS Table_De_Faits,
'Id_Diagnostic' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Diagnostic IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Diagnostic = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Diagnostic dg
ON f.Id_Diagnostic = dg.Id_Diagnostic
WHERE dg.Id_Diagnostic IS NULL;

SELECT
'FAIT_CONSULTATION' AS Table_De_Faits,
'Id_Etablissement' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Etablissement IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Etablissement = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Etablissement e
ON f.Id_Etablissement = e.Id_Etablissement
WHERE e.Id_Etablissement IS NULL;
```

```sql
SELECT
'FAIT_HOSPITALISATION' AS Table_De_Faits,
'Id_Patient' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Patient IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Patient = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_HOSPITALISATION f
LEFT JOIN DIM_Patient p
ON f.Id_Patient = p.Id_Patient
WHERE p.Id_Patient IS NULL;

SELECT
'FAIT_HOSPITALISATION' AS Table_De_Faits,
'Id_Etablissement' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Etablissement IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Etablissement = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_HOSPITALISATION f
LEFT JOIN DIM_Etablissement e
ON f.Id_Etablissement = e.Id_Etablissement
WHERE e.Id_Etablissement IS NULL;

SELECT
'FAIT_HOSPITALISATION' AS Table_De_Faits,
'Id_Diagnostic' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Diagnostic IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Diagnostic = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_HOSPITALISATION f
LEFT JOIN DIM_Diagnostic dg
ON f.Id_Diagnostic = dg.Id_Diagnostic
WHERE dg.Id_Diagnostic IS NULL;

SELECT
'FAIT_HOSPITALISATION' AS Table_De_Faits,
'Id_Date_Entree' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Date_Entree IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Date_Entree = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_HOSPITALISATION f
LEFT JOIN DIM_Temps d
ON f.Id_Date_Entree = d.Id_Temps
WHERE d.Id_Temps IS NULL;

SELECT
'FAIT_DECES' AS Table_De_Faits,
'Id_Temps' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Temps IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Temps = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_DECES f
LEFT JOIN DIM_Temps d
ON f.Id_Temps = d.Id_Temps
WHERE d.Id_Temps IS NULL;
```

Résultats :

Table de faits	Clé	Dimension	Total lignes	Références invalides	Dont NULL	Dont -1	% invalides
FAIT_ SATISFACTION	Id_ Etablissement	DIM_ Etablissement	2 080	474	0	474	22,79 %
FAIT_ SATISFACTION	Id_Region	DIM_Region_ INSEE	2 080	474	0	474	22,79 %
FAIT_ CONSULTATION	Id_Patient	DIM_Patient	1 027 136	1	0	0	0,0001 %
FAIT_ CONSULTATION	Id_Temps	DIM_Temps	1 027 136	0	0	0	0 %
FAIT_ CONSULTATION	Id_ Professionnel	DIM_ Professionnel	1 027 136	0	0	0	0 %
FAIT_ CONSULTATION	Id_Diagnostic	DIM_ Diagnostic	1 027 136	57	0	0	0,0055 %
FAIT_ CONSULTATION	Id_ Etablissement	DIM_ Etablissement	1 027 136	836 481	0	836 481	81,44 %
FAIT_HOSPITALI-SATION	Id_Patient	DIM_Patient	2 478	1	0	0	0,04 %
FAIT_HOSPITALI-SATION	Id_ Etablissement	DIM_ Etablissement	2 478	1 589	0	1 589	64,12 %
FAIT_HOSPITALI-SATION	Id_Diagnostic	DIM_ Diagnostic	2 478	0	0	0	0 %
FAIT_HOSPITALI-SATION	Id_Date_ Entree	DIM_Temps	2 478	0	0	0	0 %
FAIT_DECES	Id_Temps	DIM_Temps	24 875 113	0	0	0	0 %

Les clés vers DIM_Temps, DIM_Professionnel et (presque) DIM_Patient et DIM_Diagnostic sont cohérentes.

Le problème principal est la clé établissement : 81,44 % des consultations, 64,12 % des hospitalisations et 22,79 % des réponses de satisfaction ont Id_Etablissement = -1. Ce n'est pas une clé NULL : c'est le membre « inconnu » mis par l'ETL quand l'établissement source n'a pas été retrouvé dans DIM_Etablissement. Les 474 lignes de satisfaction ont aussi Id_Region = -1.

Les autres anomalies sont isolées : 1 Id_Patient inconnu dans FAIT_CONSULTATION, 1 dans FAIT_HOSPITALISATION et 57 Id_Diagnostic inconnus dans FAIT_CONSULTATION (valeurs renseignées mais absentes de la dimension).

Répartition par année des consultations sans établissement valide (année lue dans DIM_Temps) :

```
SELECT
t.Annee,
COUNT(*) AS Total,
SUM(CASE WHEN e.Id_Etablissement IS NULL THEN 1 ELSE 0 END) AS References_Invalides,
ROUND(SUM(CASE WHEN e.Id_Etablissement IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
↪  2) AS Pct_Invalides
FROM FAIT_CONSULTATION f
LEFT JOIN DIM_Etablissement e
ON f.Id_Etablissement = e.Id_Etablissement
LEFT JOIN DIM_Temps t
ON f.Id_Temps = t.Id_Temps
GROUP BY t.Annee
ORDER BY t.Annee;
```

Annee	Total	References_Invalides	Pct_Invalides
2015	33 895	27 640	81.55
2016	184 305	150 066	81.42
2017	133 401	108 940	81.66
2018	160 369	130 443	81.34
2019	87 495	71 345	81.54
2020	162 776	132 545	81.43
2021	145 881	118 780	81.42
2022	101 986	82 869	81.26
2023	17 028	13 853	81.35

Le taux est stable autour de 81 % pour toutes les années, 2019 et 2020 comprises (81,54 % et 81,43 %). Le défaut ne vient donc pas d'un chargement particulier : il vient de la correspondance entre les établissements de la source des consultations et DIM_Etablissement.

Contrôle de la clé temps de FAIT_DECES (nature des références invalides, puis vérification par une autre méthode) :

```
SELECT
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Temps IS NULL OR f.Id_Temps = -1 THEN 1 ELSE 0 END) AS
↪  Dont_Null_Ou_Moins_Un,
SUM(CASE WHEN f.Id_Temps > 0 AND (f.Id_Temps % 100 = 0 OR FLOOR(f.Id_Temps / 100) %
↪  100 = 0) THEN 1 ELSE 0 END) AS Dont_Jour_Ou_Mois_00,
SUM(CASE WHEN f.Flag_Date_Partielle = true THEN 1 ELSE 0 END) AS
↪  Dont_Flag_Date_Partielle,
MIN(f.Id_Temps) AS Id_Temps_Min,
MAX(f.Id_Temps) AS Id_Temps_Max
FROM FAIT_DECES f
LEFT JOIN DIM_Temps d
ON f.Id_Temps = d.Id_Temps
WHERE d.Id_Temps IS NULL;

SELECT COUNT(*) AS Deces_Avec_Date_Connue FROM FAIT_DECES f LEFT SEMI JOIN DIM_Temps
↪  d ON (f.Id_Temps = d.Id_Temps);

SELECT
COUNT(*) AS Total,
SUM(CASE WHEN Id_Temps % 100 = 0 OR FLOOR(Id_Temps / 100) % 100 = 0 THEN 1 ELSE 0
↪  END) AS Jour_Ou_Mois_00,
SUM(CASE WHEN Flag_Date_Partielle = true THEN 1 ELSE 0 END) AS
↪  Flag_Date_Partielle_Vrai,
SUM(CASE WHEN Id_Temps < 18800102 OR Id_Temps > 20301231 THEN 1 ELSE 0 END) AS
↪  Hors_Plage_DIM_Temps
FROM FAIT_DECES;
```

References_ Invalides	Deces_Avec_ Date_Connue	Total	Jour_Ou_ Mois_00	Flag_Date_ Partielle_Vrai	Hors_Plage_ DIM_Temps
0	24 875 113	24 875 113	0	89 563	0

Dans l'entrepôt actuel, les 24 875 113 décès ont un Id_Temps présent dans DIM_Temps (1880-01-02 à 2030-12-31). Les 89 563 décès à date partielle ont tous reçu une date valide. Il n'y a donc pas de décès hors de la dimension temps dans les données chargées aujourd'hui.

3.3.1 Attention : référentiel des régions de FAIT_DECES

DIM_Region_INSEE a une ligne par commune (39 520 lignes). Id_Region est une clé de substitution de 1 à 39 520, Code_Commune_INSEE est le code INSEE (39 520 valeurs distinctes) et Code_Region le code de région (19 valeurs). Dans FAIT_DECES, Id_Region_Deces va de -1 à 39 514 et Id_Region_Naissance de -1 à 39 512. Pour savoir quel référentiel est utilisé, on compte les correspondances avec chacune des trois colonnes :

```
SELECT
'Id_Region_Deces' AS Colonne,
COUNT(*) AS Total,
SUM(CASE WHEN f.Id_Region_Deces IS NULL OR f.Id_Region_Deces = -1 THEN 1 ELSE 0 END)
↪  AS Null_Ou_Moins_Un,
SUM(CASE WHEN r1.Id_Region IS NOT NULL THEN 1 ELSE 0 END) AS Correspond_Id_Region,
SUM(CASE WHEN r2.Code_Commune IS NOT NULL THEN 1 ELSE 0 END) AS
↪  Correspond_Code_Commune_INSEE,
SUM(CASE WHEN r3.Code_Reg IS NOT NULL THEN 1 ELSE 0 END) AS Correspond_Code_Region
FROM FAIT_DECES f
LEFT JOIN DIM_Region_INSEE r1
ON f.Id_Region_Deces = r1.Id_Region
LEFT JOIN (SELECT DISTINCT CAST(Code_Commune_INSEE AS INT) AS Code_Commune FROM
↪  DIM_Region_INSEE) r2
ON f.Id_Region_Deces = r2.Code_Commune
LEFT JOIN (SELECT DISTINCT CAST(Code_Region AS INT) AS Code_Reg FROM
↪  DIM_Region_INSEE) r3
ON f.Id_Region_Deces = r3.Code_Reg;

SELECT
'Id_Region_Naissance' AS Colonne,
COUNT(*) AS Total,
SUM(CASE WHEN f.Id_Region_Naissance IS NULL OR f.Id_Region_Naissance = -1 THEN 1 ELSE
↪  0 END) AS Null_Ou_Moins_Un,
SUM(CASE WHEN r1.Id_Region IS NOT NULL THEN 1 ELSE 0 END) AS Correspond_Id_Region,
SUM(CASE WHEN r2.Code_Commune IS NOT NULL THEN 1 ELSE 0 END) AS
↪  Correspond_Code_Commune_INSEE,
SUM(CASE WHEN r3.Code_Reg IS NOT NULL THEN 1 ELSE 0 END) AS Correspond_Code_Region
FROM FAIT_DECES f
LEFT JOIN DIM_Region_INSEE r1
ON f.Id_Region_Naissance = r1.Id_Region
LEFT JOIN (SELECT DISTINCT CAST(Code_Commune_INSEE AS INT) AS Code_Commune FROM
↪  DIM_Region_INSEE) r2
ON f.Id_Region_Naissance = r2.Code_Commune
LEFT JOIN (SELECT DISTINCT CAST(Code_Region AS INT) AS Code_Reg FROM
↪  DIM_Region_INSEE) r3
ON f.Id_Region_Naissance = r3.Code_Reg;
```

Colonne	Total	Null_Ou_Moins_Un	Correspond_Id_Region	Correspond_Code_Commune_INSEE	Correspond_Code_Region
Id_Region_Deces	24 875 113	12 810	24 862 303	8 880 782	29 290
Id_Region_Naissance	24 875 113	115 752	24 759 361	10 054 428	30 246

Toutes les valeurs différentes de -1 correspondent à un Id_Region (12 810 + 24 862 303 = 24 875 113 et 115 752 + 24 759 361 = 24 875 113). Les correspondances avec Code_Commune_INSEE et Code_Region ne sont que des coïncidences de nombres. Id_Region_Deces et Id_Region_Naissance utilisent donc bien le même référentiel que Id_Region : la clé de substitution de DIM_Region_INSEE, au niveau commune.

Le contrôle de clé étrangère est donc fait contre Id_Region :

```
SELECT
'FAIT_DECES' AS Table_De_Faits,
'Id_Region_Deces' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Region_Deces IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Region_Deces = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_DECES f
LEFT JOIN DIM_Region_INSEE r
ON f.Id_Region_Deces = r.Id_Region
WHERE r.Id_Region IS NULL;

SELECT
'FAIT_DECES' AS Table_De_Faits,
'Id_Region_Naissance' AS Cle_Etrangere,
COUNT(*) AS References_Invalides,
SUM(CASE WHEN f.Id_Region_Naissance IS NULL THEN 1 ELSE 0 END) AS Dont_Null,
SUM(CASE WHEN f.Id_Region_Naissance = -1 THEN 1 ELSE 0 END) AS Dont_Moins_Un
FROM FAIT_DECES f
LEFT JOIN DIM_Region_INSEE r
ON f.Id_Region_Naissance = r.Id_Region
WHERE r.Id_Region IS NULL;
```

Table de faits	Clé	Dimension	Total lignes	Références invalides	Dont NULL	Dont -1	% invalides
FAIT_DECES	Id_Region_Deces	DIM_Region_INSEE	24 875 113	12 810	0	12 810	0,05 %
FAIT_DECES	Id_Region_Naissance	DIM_Region_INSEE	24 875 113	115 752	0	115 752	0,47 %

Les références invalides sont uniquement des -1 : commune de décès ou de naissance inconnue. Le taux de naissance inconnue est plus élevé, ce qui peut s'expliquer par des naissances à l'étranger ou dans des communes qui n'existent plus (hypothèse non vérifiée).

3.4 RÉSULTATS ATTENDUS

Les contrôles permettront de déterminer :

- Le nombre de références valides.
- Le nombre de références invalides.
- Les tables concernées par les éventuelles incohérences.

Les références invalides devront être analysées afin de déterminer si elles résultent d'une donnée source manquante, d'une erreur d'intégration ou d'une correspondance absente dans une dimension.

Indicateur	Résultat
Relations contrôlées	14 (les 12 du tableau + les 2 clés régionales de FAIT_DECES)
Relations sans aucune référence invalide	5 : FAIT_CONSULTATION.Id_Temps, FAIT_CONSULTATION.Id_Professionnel, FAIT_HOSPITALISATION.Id_Diagnostic, FAIT_HOSPITALISATION.Id_Date_Entree, FAIT_DECES.Id_Temps
Références invalides (12 relations du tableau)	839 077
Références invalides (avec les 2 clés régionales de FAIT_DECES)	967 639
Dont clés égales à -1 (membre « inconnu »)	967 580
Dont valeurs renseignées absentes de la dimension	59 (1 + 1 Id_Patient, 57 Id_Diagnostic)
Dont clés NULL	0
Tables concernées	FAIT_CONSULTATION (Id_Etablissement surtout), FAIT_HOSPITALISATION (Id_Etablissement), FAIT_SATISFACTION (Id_Etablissement, Id_Region), FAIT_DECES (régions)

Origine des références invalides :

- Clés à -1 : correspondance absente dans une dimension. L'ETL n'a pas trouvé l'établissement (ou la commune) de la source dans la dimension et a mis la valeur « inconnu ». Pour les établissements, la cause probable est la clé de rapprochement : le FINESS géographique est vide dans 99,4 % de DIM_Etablissement (voir 3.2).
- 59 clés renseignées mais absentes : donnée source sans équivalent dans la dimension (erreur ponctuelle de la source).
- Aucune clé NULL : il n'y a pas d'erreur d'intégration qui perd des clés.

Pour garder un schéma en étoile propre (Kimball), il est conseillé d'ajouter dans chaque dimension une ligne « Inconnu » d'identifiant -1. Les faits resteraient alors tous reliés à une ligne de dimension.

4 DÉTECTION ET TRAITEMENT DES DOUBLONS

4.1 MÉTHODOLOGIE

La détection des doublons consiste à identifier les enregistrements possédant un même identifiant alors que celui-ci est censé être unique.

Les contrôles sont principalement réalisés sur les identifiants des dimensions et des tables de faits.

4.2 REQUÊTES DE DÉTECTION

Exemple : DIM_Etablissement

```
SELECT
    Id_Etablissement,
    COUNT(*) AS Nombre
FROM DIM_Etablissement
GROUP BY Id_Etablissement
HAVING COUNT(*) > 1;
```

Cette requête identifie les établissements possédant plusieurs enregistrements avec le même identifiant.

Autres contrôles à effectuer :

- DIM_Professionnel

```
SELECT Id_Professionnel, COUNT(*) AS Nombre
FROM DIM_Professionnel
GROUP BY Id_Professionnel
HAVING COUNT(*) > 1;
```

- DIM_Patient

```
SELECT Id_Patient, COUNT(*) AS Nombre
FROM DIM_Patient
GROUP BY Id_Patient
HAVING COUNT(*) > 1;
```

- DIM_Diagnostic

```
SELECT Id_Diagnostic, COUNT(*) AS Nombre
FROM DIM_Diagnostic
GROUP BY Id_Diagnostic
HAVING COUNT(*) > 1;
```

- FAIT_SATISFACTION

```
SELECT Id_Satisfaction, COUNT(*) AS Nombre
FROM FAIT_SATISFACTION
GROUP BY Id_Satisfaction
HAVING COUNT(*) > 1;
```

- FAIT_DECES

```sql
SELECT Id_Deces, COUNT(*) AS Nombre
FROM FAIT_DECES
GROUP BY Id_Deces
HAVING COUNT(*) > 1;
```

- FAIT_CONSULTATION

```sql
SELECT num_consultation, COUNT(*) AS Nombre
FROM FAIT_CONSULTATION
GROUP BY num_consultation
HAVING COUNT(*) > 1;
```

- FAIT_HOSPITALISATION

```sql
SELECT Id_Hospitalisation, COUNT(*) AS Nombre
FROM FAIT_HOSPITALISATION
GROUP BY Id_Hospitalisation
HAVING COUNT(*) > 1;
```

Pour DIM_Region_INSEE, le contrôle doit être adapté à la granularité de la dimension : Id_Region peut être répété si plusieurs communes appartiennent à une même région. Il est donc préférable de contrôler également Code_Commune_INSEE.

Les requêtes du rapport sont reprises et englobées dans un `SELECT COUNT(*)` pour obtenir un nombre. `Identifiants_Dupliques` est le nombre d'identifiants présents plusieurs fois, `Lignes_Concernees` le nombre de lignes qui portent ces identifiants. DIM_Temps est ajoutée en complément. Pour DIM_Region_INSEE, le contrôle est fait sur Id_Region et sur Code_Commune_INSEE, comme demandé.

```sql
SELECT 'DIM_Etablissement' AS Table_Controlee, 'Id_Etablissement' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Etablissement, COUNT(*) AS Nombre
    FROM DIM_Etablissement
    GROUP BY Id_Etablissement
    HAVING COUNT(*) > 1
) t;

SELECT 'DIM_Professionnel' AS Table_Controlee, 'Id_Professionnel' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Professionnel, COUNT(*) AS Nombre
    FROM DIM_Professionnel
    GROUP BY Id_Professionnel
    HAVING COUNT(*) > 1
) t;
```

```sql
SELECT 'DIM_Patient' AS Table_Controlee, 'Id_Patient' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Patient, COUNT(*) AS Nombre
    FROM DIM_Patient
    GROUP BY Id_Patient
    HAVING COUNT(*) > 1
) t;

SELECT 'DIM_Diagnostic' AS Table_Controlee, 'Id_Diagnostic' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Diagnostic, COUNT(*) AS Nombre
    FROM DIM_Diagnostic
    GROUP BY Id_Diagnostic
    HAVING COUNT(*) > 1
) t;

SELECT 'DIM_Temps' AS Table_Controlee, 'Id_Temps' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Temps, COUNT(*) AS Nombre
    FROM DIM_Temps
    GROUP BY Id_Temps
    HAVING COUNT(*) > 1
) t;

SELECT 'DIM_Region_INSEE' AS Table_Controlee, 'Id_Region' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Region, COUNT(*) AS Nombre
    FROM DIM_Region_INSEE
    GROUP BY Id_Region
    HAVING COUNT(*) > 1
) t;

SELECT 'DIM_Region_INSEE' AS Table_Controlee, 'Code_Commune_INSEE' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Code_Commune_INSEE, COUNT(*) AS Nombre
    FROM DIM_Region_INSEE
    GROUP BY Code_Commune_INSEE
    HAVING COUNT(*) > 1
) t;
```

```sql
SELECT 'FAIT_SATISFACTION' AS Table_Controlee, 'Id_Satisfaction' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Satisfaction, COUNT(*) AS Nombre
    FROM FAIT_SATISFACTION
    GROUP BY Id_Satisfaction
    HAVING COUNT(*) > 1
) t;

SELECT 'FAIT_DECES' AS Table_Controlee, 'Id_Deces' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Deces, COUNT(*) AS Nombre
    FROM FAIT_DECES
    GROUP BY Id_Deces
    HAVING COUNT(*) > 1
) t;

SELECT 'FAIT_CONSULTATION' AS Table_Controlee, 'num_consultation' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT num_consultation, COUNT(*) AS Nombre
    FROM FAIT_CONSULTATION
    GROUP BY num_consultation
    HAVING COUNT(*) > 1
) t;

SELECT 'FAIT_HOSPITALISATION' AS Table_Controlee, 'Id_Hospitalisation' AS Identifiant,
COUNT(*) AS Identifiants_Dupliques,
COALESCE(SUM(t.Nombre), 0) AS Lignes_Concernees
FROM (
    SELECT Id_Hospitalisation, COUNT(*) AS Nombre
    FROM FAIT_HOSPITALISATION
    GROUP BY Id_Hospitalisation
    HAVING COUNT(*) > 1
) t;
```

Table	Identifiant	Identifiants_Dupliques	Lignes_ Concernees
DIM_Etablissement	Id_Etablissement	0	0
DIM_Professionnel	Id_Professionnel	0	0
DIM_Patient	Id_Patient	0	0
DIM_Diagnostic	Id_Diagnostic	0	0
DIM_Temps	Id_Temps	0	0
DIM_Region_INSEE	Id_Region	0	0
DIM_Region_INSEE	Code_Commune_ INSEE	0	0
FAIT_SATISFACTION	Id_Satisfaction	0	0
FAIT_DECES	Id_Deces	0	0
FAIT_CONSULTATION	num_consultation	0	0
FAIT_ HOSPITALISATION	Id_Hospitalisation	0	0

Aucun identifiant n'est dupliqué, ni dans les dimensions ni dans les tables de faits. Dans DIM_Region_INSEE, Id_Region et Code_Commune_INSEE sont tous les deux uniques : la dimension a bien une ligne par commune.

4.3 TRAITEMENT DES DOUBLONS

Plusieurs méthodes peuvent être utilisées :

- **DISTINCT** : suppression des doublons strictement identiques.
- **ROW_NUMBER()** : conservation d'un seul enregistrement selon une règle de priorité.
- **tUniqRow dans Talend** : détection et filtrage des doublons pendant le processus ETL.

La suppression doit être réalisée uniquement après identification de la nature du doublon et vérification des règles métier.

Comme aucun doublon n'a été trouvé, aucun traitement n'est nécessaire. La méthode ROW_NUMBER() est quand même écrite et testée sur FAIT_CONSULTATION, pour pouvoir la reprendre après un futur chargement. On numérote les lignes de chaque identifiant et on garde `rn = 1`. Le résultat est seulement lu par un SELECT : aucune table n'est écrasée.

```
SELECT
COUNT(*) AS Lignes_Avant,
SUM(CASE WHEN t.rn = 1 THEN 1 ELSE 0 END) AS Lignes_Apres
FROM (
    SELECT
    f.*,
    ROW_NUMBER() OVER (PARTITION BY f.num_consultation ORDER BY f.Id_Temps) AS rn
    FROM FAIT_CONSULTATION f
) t;
```

Lignes_Avant	Lignes_Apres
1 027 136	1 027 136

Le nombre de lignes est le même avant et après : la requête ne retire rien, ce qui confirme l'absence de doublons. Pour appliquer réellement le traitement, il faudrait écrire ce résultat (`WHERE rn = 1`) dans une nouvelle table, puis la contrôler avant de remplacer l'ancienne.

5 SYNTHÈSE DE LA QUALITÉ DES DONNÉES

Les résultats des trois catégories de contrôles permettront de constituer un bilan global de la qualité des données intégrées.

Contrôle	Indicateur	Résultat
Complétude	Taux de valeurs renseignées par attribut	100 % pour 38 des 40 attributs contrôlés. Score (FAIT_SATISFACTION) : 78,08 % (456 NULL). Age_Deces : 99,998 % (586 NULL). En comptant aussi les chaînes vides : finess_site 0,59 % (33 711 vides), Code_Region et Nom_Region 99,09 % (358 vides), commune 99,59 % (140 vides)
Cohérence référentielle	Nombre de références invalides	839 077 sur les 12 relations du tableau (967 639 avec les 2 clés régionales de FAIT_DECES). 99,99 % sont des clés à -1, dont 836 481 Id_Etablissement de FAIT_CONSULTATION (81,44 %)
FINESS	Nombre d'identifiants au format invalide	finess_site : 33 711, tous vides (0 FINESS renseigné au mauvais format). finess_etablissement_juridique : 209, tous des codes corses 2A/2B valides. 0 FINESS NULL
Doublons	Nombre d'identifiants dupliqués	0 dans les 10 tables (11 contrôles)

Bilan : l'entrepôt est complet et sans doublon. Sa limite principale est la clé établissement dans les tables de faits, probablement liée au FINESS géographique presque toujours vide dans la source. Les analyses par établissement ne portent donc que sur environ 19 % des consultations et 36 % des hospitalisations.

6 CONCLUSION

Le contrôle de qualité des données constitue une étape essentielle dans la mise en place de l'entrepôt de données décisionnel du CHU.

Il permet de s'assurer que les données intégrées depuis les sources sont suffisamment complètes, cohérentes et fiables pour être exploitées dans les analyses décisionnelles.

Les anomalies détectées pourront être corrigées au niveau des sources, des traitements Talend ou des scripts HiveQL, selon leur origine.

Les résultats obtenus permettront ainsi de justifier la fiabilité des données intégrées et de documenter les éventuelles limites de l'entrepôt de données.